Connexion Wi-Fi avancée (ESP32)

Robotique – Grade 12 – Plan de cours de 2 heures

Enseignant : Francis Ki | Semaine du 16 au 20 novembre 2026

1 Informations générales

Titre	Connexion Wi-Fi avancée avec l'ESP32
Classe / effectif	Grade 12, 11 à 20 élèves, niveaux hétérogènes, élèves à besoins particuliers
Durée	2 heures (120 min)
Lieu	Salle informatique, un poste par élève
Matériel et logiciels	— Un poste par élève avec navigateur web. — **Wokwi** (wokwi.com, gratuit) : simulation d'un ESP32 avec Wi-Fi (réseau de test `Wokwi-GUEST`, sans mot de passe). — Facultatif : une carte ESP32 réelle par binôme et un réseau Wi-Fi 2,4 GHz de test, avec Arduino IDE configuré pour l'ESP32. — Un service Web de test accessible en HTTP, choisi et vérifié par l'enseignant la veille (par exemple une page ou une API publique qui renvoie un texte court). — Vidéoprojecteur ou tableau interactif.
Pré-requis	Séances précédentes : architecture d'un objet connecté, protocoles TCP/IP et HTTP. G11 : `setup()` et `loop()`, communication Wi-Fi de base.

Préparation avant le cours (environ 50 min)

1. Créer dans Wokwi un projet ESP32 modèle avec le code de connexion de l'annexe A et vérifier que la connexion à `Wokwi-GUEST` réussit.
2. Choisir et tester le service Web de test (adresse, format de la réponse). Prévoir un plan B si le service est indisponible.
3. Si des cartes réelles sont utilisées : la plupart des ESP32 n'utilisent que le Wi-Fi 2,4 GHz et ne se connectent pas aux réseaux d'entreprise avec identifiant individuel ; prévoir un point d'accès dédié.
4. Préparer les fichiers des annexes C à E et le formulaire de sortie.

2 Objectifs d'apprentissage

Objectifs du programme

— Configurer une connexion réseau.

— Échanger des données via Wi-Fi.

À la fin de la séance, l'élève est capable :

— d'expliquer les étapes d'une connexion Wi-Fi (SSID, mot de passe, adresse IP attribuée par DHCP) ;

— d'écrire ou de compléter un programme ESP32 qui se connecte, affiche son adresse IP et sa puissance de signal (RSSI) ;

— de gérer un échec de connexion (délai maximal, nouvelle tentative) et d'effectuer une requête HTTP simple.

Compétences numériques : lire un moniteur série, diagnostiquer une connexion, gérer les erreurs réseau.

3 Déroulé minuté (120 min)

Minutes	Étape	Contenu	Organisation
0 – 10	Accroche	Pourquoi mon téléphone se reconnecte tout seul ?	Classe entière
10 – 30	Mini-leçon	Étapes d'une connexion, DHCP, RSSI, gestion d'erreur	Classe entière
30 – 60	Pratique guidée	Connexion d'un ESP32 simulé en collectif, puis requête HTTP en binômes	Collectif puis binômes
60 – 95	Pratique autonome	Trois niveaux de besoin	Individuel sur poste
95 – 110	Mise en commun	Valoriser chaque groupe	Classe entière
110 – 120	Exit ticket	Trois questions sur poste	Individuel sur poste

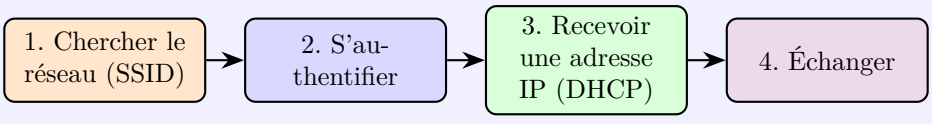
1. Accroche : le téléphone qui se reconnecte — 0 – 10 min

1. (5 min) *Que se passe-t-il quand ton téléphone arrive à la maison ? Qui lui donne une adresse ? Que se passe-t-il quand le signal est faible ?* Noter les mots : réseau, mot de passe, adresse, signal.
2. (5 min) Question-problème : « **Comment un petit microcontrôleur, sans écran, se connecte-t-il à un réseau et reste-t-il connecté ? »**

2. Mini-leçon : se connecter et rester connecté — 10 – 30 min

Support : annexe A. Quatre temps de 5 min.

Temps 1 : les étapes d'une connexion.



Temps 2 : le vocabulaire.

Terme	Sens
SSID	Nom du réseau Wi-Fi
DHCP	Service qui attribue automatiquement une adresse IP
Passerelle	Équipement qui relie le réseau local à Internet (la box)
RSSI	Puissance du signal reçu (en dBm, plus proche de 0 = plus fort)
Mode STA / AP	L'ESP32 se connecte à un réseau (STA) ou crée son propre réseau (AP)

Temps 3 : le programme. Se connecter avec un délai maximal.

Connexion avec délai maximal

```
#include <WiFi.h>

const char* ssid = "Wokwi-GUEST";
const char* mdp  = "";

void setup() {
  Serial.begin(115200);
  WiFi.mode(WIFI_STA);
  WiFi.begin(ssid, mdp);

  int essais = 0;
  while (WiFi.status() != WL_CONNECTED && essais < 20) {
    delay(500);
    Serial.print(".");
    essais++;
  }
  if (WiFi.status() == WL_CONNECTED) {
    Serial.println("\nConnecte !");
    Serial.println(WiFi.localIP());
    Serial.println(WiFi.RSSI());
  } else {
    Serial.println("\nEchec de connexion");
  }
}

void loop() { }
```

Temps 4 : rester connecté. Un réseau peut disparaître. Un programme robuste vérifie l'état de la connexion dans `loop` et se reconnecte si besoin. Ne jamais écrire un mot de passe réel dans un code partagé.

Différenciation et besoins particuliers

Colorer les étapes de la connexion de la même façon dans le schéma et dans le code. Lire le code à voix haute avec les noms des variables. Fournir un lexique bilingue (SSID, gateway).

3. Pratique guidée — 30 – 60 min

Fiche : annexe B. Programmation en binômes (pilote et navigateur, rôles échangés à mi-temps).
3a. Connexion en collectif (12 min). Ouvrir le projet Wokwi, lancer la simulation, lire dans

le moniteur série les points d'attente, l'adresse IP et le RSSI. Changer le nom du réseau pour provoquer un échec et observer le message.

3b. Requête HTTP en binômes (18 min). Compléter le programme pour envoyer une requête GET au service de test et afficher le code de statut et le début de la réponse. Alternance des rôles.

Étayage progressif de l'aide.

Niveau d'aide	Contenu	Qui intervient
Aide 1	Le programme de connexion du 3a et la liste des instructions	Le binôme seul
Aide 2	Questions guides : *Quelle instruction lance la connexion ? Quelle condition fait attendre ? Que renvoie le serveur ?*	Le binôme avec la carte d'aide
Aide 3	L'enseignant écrit l'instruction de requête avec le binôme, qui complète	L'enseignant

Différenciation et besoins particuliers

Rappeler que la simulation ne remplace pas un vrai test : le signal, les pannes et la sécurité réels ne sont pas identiques. Faire noter dans la fiche ce que le simulateur ne montre pas.

4. Pratique autonome différenciée — 60 – 95 min

Consigne commune : chaque élève ouvre le fichier de son groupe, le renomme avec son prénom et travaille seul sur son poste. L'enseignant passe d'abord auprès du groupe A pendant 10 min.

Groupe	Tâche sur poste	Annexe
A : en dessous du niveau	Programme de connexion à compléter (instructions manquantes signalées par des tirets, banque de mots : `WiFi.begin`, `WL_CONNECTED`, `localIP`, `RSSI`). Lire la sortie du moniteur série et la décrire.	Annexe C
B : au niveau	Écrire seul un programme qui se connecte avec un délai maximal, affiche l'adresse IP et le RSSI, puis fait une requête HTTP au service de test et affiche le code de statut. Fournir un tableau de tests (réseau correct, réseau inexistant).	Annexe D
C : au-dessus du niveau	Ajouter une reconnexion automatique dans `loop`, un compteur de tentatives et un message d'état. Comparer l'attribution d'une adresse par DHCP et une adresse fixe (`WiFi.config`), et discuter la sécurité d'un réseau ouvert.	Annexe E

Plan B si la simulation est indisponible : analyser le code et la sortie du moniteur série imprimés (annexe A) et répondre aux questions.

5. Mise en commun — 95 – 110 min

1. (2 min) Retour au calme, écrans mis en veille.
2. (9 min) Trois passages courts de 3 min, avec valorisation :
   - **Groupe A** : un élève lit la sortie du moniteur et identifie l'adresse IP. Valoriser la lecture technique.
   - **Groupe B** : un binôme présente son tableau de tests (réseau correct, réseau inexistant). Valoriser la gestion de l'échec.
   - **Groupe C** : un élève présente sa reconnexion automatique et la comparaison DHCP et adresse fixe. Valoriser la profondeur.
3. (4 min) Conclusion : *se connecter à un réseau demande un SSID, une authentification et une adresse IP ; un bon programme limite l'attente, gère l'échec et se reconnecte.*

6. Clôture : exit ticket — 110 – 120 min

Formulaire en ligne ou papier (annexe G) :

1. À quoi sert le DHCP ?
2. Pourquoi ajoute-t-on un nombre maximal d'essais dans la boucle d'attente ?
3. Auto-évaluation : *Je maîtrise / Je progresse / J'ai besoin d'aide.*

Lire les réponses avant la séance suivante pour ajuster les groupes de besoin.

Annexe A : Support résumé de la mini-leçon

Connexion Wi-Fi avec l'ESP32

Nom : _____ Date : _____

SSID → authentification → adresse IP (DHCP) → échanges. Toujours limiter l'attente et gérer l'échec.

Connexion et requête HTTP (modèle)

```
#include <WiFi.h>
#include <HTTPClient.h>

void loop() {
  if (WiFi.status() == WL_CONNECTED) {
    HTTPClient http;
    http.begin("http://adresse-du-service-de-test");
    int code = http.GET();
    Serial.println(code);
    if (code > 0) {
      Serial.println(http.getString());
    }
    http.end();
  } else {
    Serial.println("Reconnexion...");
    WiFi.reconnect();
  }
  delay(10000);
}
```

Instruction	Rôle
`WiFi.begin(ssid, mdp)`	Lance la connexion
`WiFi.status()`	Donne l'état (WL_CONNECTED si connecté)
`WiFi.localIP()`	Adresse IP reçue
`WiFi.RSSI()`	Puissance du signal
`http.GET()`	Envoie une requête GET, renvoie le code de statut

Annexe B : Fiche de pratique guidée

Pratique guidée : se connecter et échanger Nom : _____
Date : _____

3a. Observations

Adresse IP obtenue : ..... RSSI : ..... dBm
Message avec un mauvais nom de réseau : .....

3b. Requête HTTP

Élément	Ce que j'observe
Code de statut	
Début de la réponse	
Nombre de tentatives de connexion	

Cartes d'aide à utiliser dans l'ordre

Aide 1 : regarde le programme de connexion et le tableau des instructions.
Aide 2 : Quelle instruction lance la connexion ? Quelle condition fait attendre ? Que renvoie le serveur ?
Aide 3 : demande à l'enseignant d'écrire l'instruction de requête avec toi.

Annexe C : Niveau A (en dessous du niveau)

Niveau A : je complète la connexion

Nom : _____ Date : _____

Banque : WiFi.begin, WL_CONNECTED, localIP, RSSI.

Programme à compléter

```
WiFi._____(ssid, mdp);
while (WiFi.status() != _____) {
  delay(500);
}
Serial.println(WiFi._____());
Serial.println(WiFi._____());
```

Activité : lis la sortie du moniteur

Sortie	Ce que cela signifie
`....`	
`192.168.1.25`	
`-60`	
`Echec de connexion`	

Vrai ou faux ?

1. Le DHCP attribue une adresse IP. ☐ Vrai ☐ Faux
2. Un RSSI de −30 est un signal plus fort qu'un RSSI de −80. ☐ Vrai ☐ Faux

Annexe D : Niveau B (au niveau)

Niveau B : connexion et requête HTTP

Nom : _____

Date : _____

Cahier des charges : se connecter avec un maximum de 20 essais, afficher l'adresse IP et le RSSI, envoyer une requête GET, afficher le code de statut.

Test	Résultat attendu	Obtenu	Conforme ?
Réseau correct			
Réseau inexistant			
Service injoignable			

Questions

1. Que fait ton programme si la connexion échoue ? .....
2. Quel code HTTP indique un succès ? .....

Annexe E : Niveau C (au-dessus du niveau)

Niveau C : robustesse et sécurité

Nom : _____ Date : _____

1. Ajoute dans `loop` une vérification de l'état et une reconnexion automatique avec un compteur de tentatives.
2. Compare une adresse attribuée par DHCP et une adresse fixe (`WiFi.config`) :

Critère	DHCP	Adresse fixe
Simplicité		
Risque de conflit		
Usage typique		

3. Pourquoi un réseau Wi-Fi ouvert est-il risqué pour un objet connecté ? _____

_____

Défi bonus : propose une façon de ne pas écrire le mot de passe directement dans le code.

Annexe F : Corrections et attendus (enseignant)

Annexe B

Adresse IP : une adresse privée attribuée par le simulateur (de la forme 10.x.x.x ou 192.168.x.x). RSSI : valeur négative (par exemple -60). Avec un mauvais nom : « Echec de connexion » après les essais. Requête GET réussie : code 200.

Annexe C

`begin`, `WL_CONNECTED`, `localIP`, `RSSI`. Sorties : points d'attente de la connexion ; adresse IP reçue ; puissance du signal (assez bonne) ; échec après le nombre maximal d'essais. Vrai, Vrai.

Annexe D

Réseau correct : IP et RSSI affichés, code 200. Réseau inexistant : « Echec » sans bloquer. Service injoignable : code négatif ou erreur, programme qui continue.

Annexe E

Reconnexion : `if (WiFi.status() != WL_CONNECTED) { WiFi.reconnect(); tentatives++; }`. DHCP : simple, adresse change ; fixe : adresse stable, risque de conflit. Réseau ouvert : messages lisibles par d'autres, pas d'authentification. Mot de passe : fichier séparé non partagé, ou saisie au démarrage. **Critères de réussite :** connexion avec délai, gestion d'échec, requête HTTP correcte.

Annexe G : Exit ticket et grille d'observation

Exit ticket : connexion Wi-Fi Nom : _____ Date : _____

1. À quoi sert le DHCP ?

2. Pourquoi ajoute-t-on un nombre maximal d'essais dans la boucle d'attente ?

3. Je me sens : ☐ Je maîtrise ☐ Je progresse ☐ J'ai besoin d'aide

Attendus : 1. À attribuer automatiquement une adresse IP. 2. Pour ne pas rester bloqué indéfiniment si le réseau est introuvable.

Grille d'observation (enseignant)

Élève	Connexion réussie	Échec géré	Requête HTTP	Groupe suivant